Visitor Pattern File System

Assignment 06 Software Design Patterns

GitHub repository: _____

Purpose and design

This Java 17 Maven project models an in-memory file system and applies three independent operations to the same tree: total size, configurable file search and structural statistics. The Visitor pattern keeps these operations in separate classes, so another operation can be added without editing the existing file and directory classes. The implementation follows requirements R1-R8 and is verified with JUnit 5 [1, 2].

Element declares getName() and accept(Visitor). AbstractNode stores a fixed name and its parent. FileNode adds a non-negative long byte size. DirectoryNode stores child nodes and provides add() and an immutable snapshot through getChildren(). The UML appendix shows these classes and the three visitors.

DirectoryNode.add() checks for null, cycles, an existing parent and duplicate sibling names before changing the tree. A cycle is detected by walking from the target directory through its ancestors and comparing object identity with the proposed child. Invalid changes throw IllegalArgumentException. Names are case-sensitive single path components; names and file sizes cannot be changed after construction.

Double dispatch and traversal

First, element.accept(visitor) dynamically selects the concrete element method. Inside FileNode.accept(), this has the compile-time type FileNode, so visitor.visit(this) selects the visit(FileNode) overload. The actual visitor object then determines which implementation runs at runtime. DirectoryNode follows the same two-step process with visit(DirectoryNode). No runtime type checks or casts select behavior [2].

Visitors own recursive depth-first traversal. Each visit(DirectoryNode) calls accept(this) once for every child; accept() itself only dispatches. The single-parent, acyclic tree ensures that each reachable node is visited once per operation. Empty directories are still visited.

Paths depth and reusable state

SearchVisitor tests the filename predicate and size >= minimumSize together, then sorts the matching paths lexicographically. Paths use / and omit the selected directory root and its ancestors. A selected file root uses its own filename. StatisticsVisitor counts the root directory, starts depth at zero and updates maximum depth for both files and directories.

calculate(root), search(root) and analyze(root) reset all accumulated state before traversal. Instances can therefore be reused on the same or a different root. Direct accept() calls are the dispatch primitive; complete operations use these entry methods.

Clean Code practices

Five practices are visible in the implementation and correspond to the required design explanation [1, 3].

1 Meaningful naming. TotalSizeVisitor.calculate(), SearchVisitor.minimumSize and StatisticsVisitor.getMaxDepth() state their purpose, making the operation and its result clear without extra explanatory comments [3, p. 18].

2 Single Responsibility. FileNode and DirectoryNode represent the tree, while TotalSizeVisitor, SearchVisitor and StatisticsVisitor each implement one operation, so a change to search rules does not change the data model [3, pp. 138-139].

3 Small methods. The one-line accept() methods only dispatch, while DirectoryNode.checkCycle() and checkDuplicateName() each validate one rule, keeping individual methods easy to read and test [3, pp. 34-35].

4 Encapsulation. DirectoryNode.getChildren() returns List.copyOf(children), and AbstractNode.attachTo() is package-private, so callers must use the validated add() operation to change child ownership [3, pp. 93-95].

5 No type-checking chains. FileNode.accept() and DirectoryNode.accept() call visitor.visit(this), which routes behavior through overloaded methods instead of repeating instanceof branches in every operation [3, p. 38; 2, p. 18].

Extension and design tradeoffs

StatisticsVisitor was added in a separate incremental commit after the original size and search operations. The extension added the new visitor and its tests and updated the demo client and documentation. Element, Visitor, FileNode, DirectoryNode and their accept() methods remained unchanged. The new operation uses the original contracts.

Adding a new element type has a different cost: Visitor needs another visit overload, and every concrete visitor must define behavior for that type. Visitor therefore fits a stable set of element types with evolving operations. Visitors also need read access to node data, so getters expose only the information these operations require [2, 4].

Keeping traversal in the visitors makes this small example easy to trace, although each visitor repeats the short child loop. A shared walker could remove that duplication in a larger project. This version is single-threaded and does not support mutation during a traversal. Nesting has no fixed level limit but is bounded by the JVM call stack; a size total beyond the long range throws ArithmeticException.

References

[1] Assignment 06, Visitor Pattern - Advanced File System. Requirements R1-R8, verification T1-T8 and submission requirements.

[2] Lecture 9, The Visitor Pattern. Supplied PDF, pp. 2-4, 14, 18, 23-24 and 35-39.

[3] Martin, R. C. Clean Code. 2008. Printed pp. 18, 34-35, 38, 93-95 and 138-139.

[4] Freeman, E., Freeman, E., Sierra, K. and Bates, B. Head First Design Patterns. First edition, 2004. Printed pp. 628-629.

Execution and verification

From the project root containing pom.xml, with JDK 17 and Maven installed:

```
mvn test
mvn -q compile exec:java
```

Demo data

App.createDemoTree() builds the workspace root, nested docs/archive and images directories, and an empty directory named empty.

File path	Bytes	File path	Bytes
notes.txt	120	images/logo.png	500
docs/manual.txt	300	docs/todo.txt	80
docs/archive/old.txt	200	data.bin	1024

Actual program output

```
Visitor Pattern - In-memory File System
Total size: 2224 bytes
Search (name ends with ".txt", size >= 100):
  docs/archive/old.txt
  docs/manual.txt
  notes.txt
Files: 6
Directories (including root): 5
Maximum depth (root = 0): 3
```

Automated test results

JUnit 5 test class	Main coverage	Tests
VisitorTest	Dispatch, size, search, reuse and boundaries	11
TreeIntegrityTest	Invalid changes and protected child collections	9
StatisticsVisitorTest	Counts, depth, empty roots and reuse	5

Maven result: 25 tests run, 0 failures, 0 errors, 0 skipped. BUILD SUCCESS.

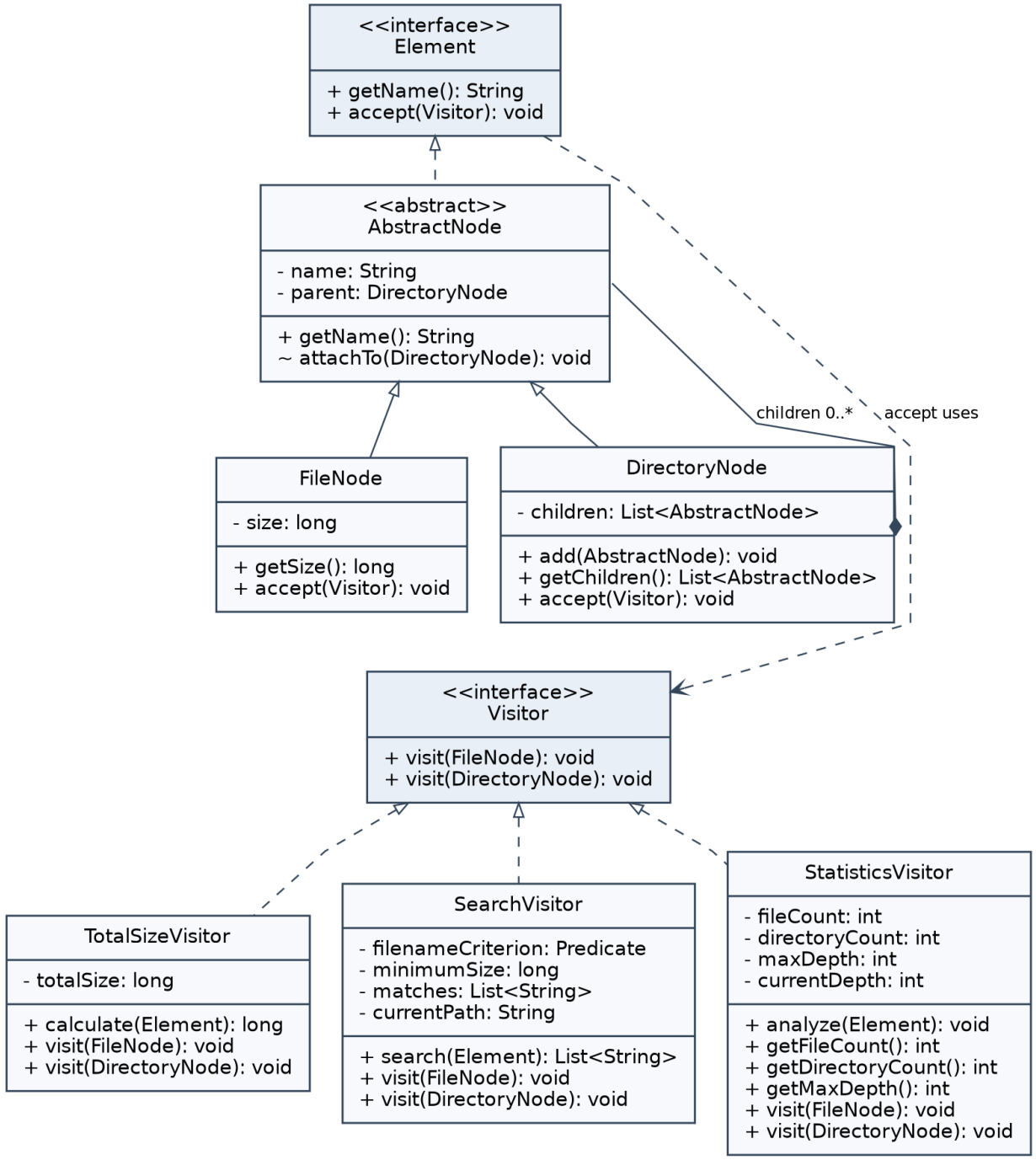
The suite covers all T1-T8 checkpoints, including sorted subtree paths, inclusive thresholds, empty directories, file roots and repeated operations.

Changing the search configuration

```
mvn -q compile exec:java "-Dexec.args=.txt 200"
```

The matching paths become docs/archive/old.txt and docs/manual.txt. Total size, file count, directory count and maximum depth remain unchanged because only the search configuration changes.

UML appendix



Hollow triangles indicate inheritance or interface implementation; dashed lines denote implementation or dependency. The filled diamond at DirectoryNode represents ownership of zero or more children. A root has no parent.